Figure 2: Looking Glass

```
    color: #ffffff;
    background-color: rgba(10,10,10,0.7);
    border-radius: 5px;
    padding: .5em;
}
```

You can change various styles associated with your extension by changing what's needed in this file.

Looking Glass

This is the tool used to debug your GNOME shell extensions and Javascript code. To open Looking Glass, press Alt+F2, type in 'lg' and hit 'Enter'. The Looking Glass window will open up.

Looking Glass is divided into tabs: Evaluator, Windows, Memory and Extensions. The Evaluator window is where you can type in random Javascript code. You can also use the picker at the left corner of Looking Glass, which you can use to select various elements of the GNOME shell and find out the element's name. The *Windows* tab shows which windows are active, while the *Extensions* tab shows the various extensions running on your system, along with any errors. If any extension runs into some error, the error will be displayed here.

Customising your extension

Let us now modify your standard 'Hello world' extension. Let's try and modify the type of icons displayed in the right panel. By default, the icons displayed in the right panel are symbolic icons, and your extension will change them to full colour icons. Let us look at the code to do this:

```
const St = imports.gi.St;
const Main = imports.ui.main;
const Tweener = imports.ui.tweener;
const PopupMenu = imports.ui.popupMenu;

function init() {

}
 function disable() {
        let children = Main.panel._rightBox.get_children();
        for (let i = 0; i < children.length; i++) {
            if (children[i] && children[i]._delegate._
iconActor) {
                children[i]._delegate._iconActor.icon_type =
St.IconType.SYMBOLIC;
                children[i]._delegate._iconActor.style_class
= 'system-status-icon';
            }
        }
    }

     function enable() {
        let children = Main.panel._rightBox.get_children();
        for (let i = 0; i < children.length; i++) {
            if (children[i] && children[i]._delegate._
iconActor) {
                children[i]._delegate._iconActor.icon_type =
St.IconType.FULLCOLOR;
                children[i]._delegate._iconActor.style_class
= 'color-status-button';
            }
        }
    }
```

For the sake of simplicity, I have left the *init* function empty, but all your initialisations should be done inside that function. In the *Enable* function, you get all elements in the right panel with the *Main.panel._rightBox.get_children()* function; then you loop over all children, changing their style class from symbolic icon to full colour. In the *Disable* function, do the same with one difference—change the icon style from full colour to symbolic.

As we have seen, the GNOME shell allows you to extend its functionality by adding extensions. Building these extensions is easier with tools like the GNOME shell extension-tool, and Looking Glass.

You can learn a lot about building extensions by reading the code of existing extensions. You will find a few of them on GNOME's GIT under a module called 'gnome-shell-extensions'. Also remember that Looking Glass is your friend. Get familiar with it and use it to solve many a coding error. Further documentation can be found on the GNOME website *https://live.gnome.org/GnomeShell/Extensions*. You have also seen how easy it is to customise the GNOME shell to suit your needs. GNOME extensions let users take control, making it easier to achieve the perfectly customised system. END

By: Sahil Chelaramani

The author is an open source activist who loves Linux, Python and Android. He is a third year student at the Goa Engineering College (GEC).